

PRIORITY QUEUE E HEAP

INDICE SEZIONE

- ☐ DEFINIZIONE
- ☐ INTERFACCIA
- ☐ APPLICAZIONI
- ☐ IMPLEMENTAZIONE TRAMITE LISTE
- ☐ IMPLEMENTAZIONE TRAMITE HEAP
- ☐ COSTRUZIONE HEAP
- ☐ SORTING CON PRIORITY QUEUE

DEFINIZIONE

Prima di dare la definizione di una coda con priorità, definiamo *entry*.

ENTRY

E' una *coppia* (*chiave, valore*), dove la chiave proviene da un dominio K ed il valore da un dominio V .

```
public interface Entry<K,V> {  
    // Returns the key of the entry  
    K getKey();  
  
    // Returns the value of the entry  
    V getValue();  
}
```

Possiamo adesso definire:

PRIORITY QUEUE

Collezione di entry le cui *chiavi* rappresentano *priorità* e provengono da un universo *totalmente ordinato* K .

Come tipo di dato astratto, la priority queue deve permettere di:

- Trovare/rimuovere la entry di *massima priorità*.

- Inserire una *nuova entry*.

NOTE

1. Le chiavi non sono necessariamente *distinte*.
2. Convenzionalmente si assuma che più *piccolo* è il valore della *chiave* e più *alta* è la sua *priorità*, ma vedremo anche utilizzi in cui vale l'opposto.

INTERFACCIA

```
public interface PriorityQueue<K,V> {  
    int size();  
  
    // Inserts and returns a new entry (key, value)  
    Entry<K,V> insert(K value, V value);  
  
    // Returns an entry with min key, without removing it  
    Entry<K,V> min();  
  
    // Returns and removes an entry with min key  
    Entry<K,V> removeMin();  
}
```

ATTENZIONE

Se esistono *più entry* con chiave minima, `min` e `removeMin` ne restituiscono (e rimuovono) una *arbitraria*.

APPLICAZIONI

Le priority queue sono impiegate nei seguenti contesti:

- Algoritmo di *Dijkstra* per trovare i cammini minimi su un grafo.
- Pattern discovery.
- *Scheduling processi* nei sistemi operativi.
- Gestione *richieste di banda* nelle reti.
- *Simulazione discreta* a eventi.

IMPLEMENTAZIONE TRAMITE LISTE

LISTE NON ORDINATE

Sia P una *lista non ordinata* di n entry.

```
PositionalList< Entry<K,V> >
```

Vediamo come implementare i metodi richiesti dall'interfaccia:

```
Algorithm min()
1:  $v \leftarrow P.first()$ 
2:  $e \leftarrow v.getElement()$ 
3: while  $P.after(v) \neq \text{null}$  do
4:    $v \leftarrow P.after(v)$ 
5:   if  $v.getElement().getKey() < e.getKey()$  then
6:      $e \leftarrow v.getElement()$ 
7:   end if
8: end while
9: return  $e$ 
```

La complessità di $\text{min}()$ è $\Theta(n)$ in quanto deve *scansionare ogni entry* per trovare quella con chiave minore.

L'algoritmo $\text{removeMin}()$ può essere implementato a partire da $\text{min}()$: si trova l'entry e con chiave minima e si fa puntare $e.previous$ a $e.after$.

Vediamo $\text{insert}()$:

```
Algorithm insert( $k, x$ )
1:  $e \leftarrow (k, x)$ 
2:  $P.addLast(e)$ 
3: return  $e$ 
```

La complessità di $\text{insert}()$ è $\Theta(1)$ in quanto possiamo inserire il nuovo elemento dove vogliamo, non avendo necessità di mantenere un ordinamento.

LISTE ORDINATE

Sia ora P una *lista ordinata* (in senso crescente) di n entry.

Vediamo come implementare i metodi in questo caso.

```
Algorithm min()
1: return  $P.first().getElement()$ 
```

La complessità di $\text{min}()$ è ora $\Theta(1)$ perchè, essendo la lista *ordinata* in senso crescente, l'elemento con *chiave minima* è il *primo*.

Per removeMin basta aggiornare il primo nodo della lista.

```

Algorithm insert( $k, x$ )
1:  $e \leftarrow (k, x)$ 
2:  $v \leftarrow P.first()$ 
3: while  $v \neq \text{null}$  do
4:   if  $v.getElement().getKey \geq k$  then
5:      $P.addBefore(v, e)$ 
6:     return  $e$ 
7:   else
8:      $v \leftarrow P.after()$ 
9:   end if
10: end while
11:  $P.insertLast(e)$ 
12: return  $e$ 

```

La complessità di `insert()` è ora $\Theta(n)$ in quanto, dovendo mantenere l'ordinamento, bisogna trovare la *posizione corretta* in cui inserire la entry.

RIEPILOGO

TIPO LISTA	Insert	min	removeMin
Non ordinata	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Ordinata	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$

Vediamo quindi che il *costo* delle operazioni è molto *sbilanciato* verso l'inserzione oppure la rimozione in base al tipo di lista scelta.

IMPLEMENTAZIONE TRAMITE HEAP

Un modo per *bilanciare* i costi delle operazioni viste in > **RIEPILOGO** consiste nell'implementare una priority queue con una struttura dati chiamata *heap*.

HEAP

MIN-HEAP (HEAP)

E' un *albero binario completo* in cui ogni *nodo* v memorizza una *entry* e soddisfa una certa proprietà.

Heap-order property: la *chiave* in v è *minore o uguale* della chiave in *ciascun figlio* di v .

Vedi > **ALBERI BINARI > ALBERI BINARI COMPLETI.**

☰ MAX-HEAP

Heap in cui la definizione della *heap-order property* cambia come segue:

$$\text{minore/uguale} \implies \text{maggiore/uguale}$$

☰ NODO LAST

Il nodo *last* in uno heap di altezza h è il nodo più a *destra* del livello h .

Se non diversamente specificato, assumeremo sempre che uno heap sia realizzato tramite un array: vedi > [ALBERI BINARI > ALBERI BINARI COMPLETI SU ARRAY](#).

PROPRIETA' HEAP

Sia P uno heap con n entry.

Dalla definizione si ricavano le seguenti proprietà:

🔥 PROPRIETA'

1. Le chiavi incontrate lungo un cammino dalla radice verso le foglie formano una *sequenza non decrescente*.
2. Per qualsiasi discendente u di un nodo $v \in P$ si ha che:
 $e_u.\text{getKey}() \geq e_v.\text{getKey}()$.
3. La *radice* contiene una entry con *chiave minima*.
4. Se le *chiavi* sono tutte *distinte*, la entry con *chiave massima* e_{max} sta in una foglia di P .

Capiamo quindi che uno heap **non** assicura un *ordinamento totale* tra le entry, ma un ordinamento lungo *percorsi radice-foglia*.

METODI PRIORITY-QUEUE SU HEAP

Sia P uno heap con n entry implementato tramite array.

MIN

Il metodo `min()` è banale:

```
Algorithm min()
1: return P[0]
```

INSERT

Vediamo ora l'**inserimento**.

IDEA

- Inseriamo la nuova entry come *successore* (nel level numbering) del nodo *last*.
- *Ricostruiamo* la *heap-order* property dello heap lungo il cammino da last alla radice.

```
Algorithm insert(k,x)
1: e ← (k,x)
2: P[++last] ← e
3: i ← last
4: while i>0 and P[⌊ $\frac{i-1}{2}$ ⌋].getKey() > P[i].getKey() do
5:   swap(P[i],P[⌊ $\frac{i-1}{2}$ ⌋])
6:   i ← ⌊ $\frac{i-1}{2}$ ⌋
7: end while
8: return e
```

Le istruzioni eseguite dal ciclo while sono dette **up-heap bubbling** perchè fanno *risalire* la nuova entry inserita come una bolla.

La **correttezza** dell'algoritmo `insert(k, x)` deriva dalla *heap-order* property **estesa**.

HEAP ORDER PROPERTY ESTESA

- La **chiave** in un nodo v è *minore o uguale* della chiave in *ciascun figlio* di v .
(BASE)
- la **chiave** in v è *maggiore o uguale* della chiave in *ciascun antenato* di v .
(ESTENSIONE)

Si dimostra che non viene mai violata la struttura di albero binario completo e, ad ogni iterazione del ciclo, le *uniche violazioni* della heap order *property estesa* possono essere tra $P[i]$ ed un suo antenato: cioè la proprietà vale per tutti i figli del nodo e, al termine del ciclo, per tutto il percorso radice-foglia.

Per quanto riguarda la complessità, consideriamo l'esecuzione di `insert()` su uno heap P di n nodi ed altezza $h = \lfloor \log_2(n + 1) \rfloor$ dopo l'inserimento:

- La complessità è proporzionale al *numero di iterazioni del ciclo* while, dato che questo esegue un numero $\Theta(1)$ operazioni.
- Numero di iterazioni del while $\leq h$.
- Esiste un'istanza che richiede *esattamente h operazioni* (si inserisce una entry con chiave minore di tutte quelle presenti).

Concludiamo quindi che la complessità è $\in \Theta(h) \in \Theta(\log n)$.

REMOVE MIN

IDEA

- *Rimuoviamo* la entry nella radice.
- *Mettiamo* le entry $P[\text{last}]$ nella radice.
- *Ricostruiamo* la heap order property dello heap a partire dalla radice verso le foglie.

Sia `indexMinChild(P, i)` un metodo che restituisce l'indice del figlio di $P[i]$ con chiave minima ($2i + 1$ o $2i + 2$) se $P[i]$ è un nodo interno oppure null se $P[i]$ è foglia.

Algorithm `removeMin()`

```
1: minentry  $\leftarrow P[0]$ 
2:  $P[0] \leftarrow P[\text{last}--]$ 
3:  $i \leftarrow 0$ 
4:  $j \leftarrow \text{indexMinChild}(P, i)$ 
5: while  $j \neq \text{null}$  and  $P[i].\text{getKey}() > P[j].\text{getKey}()$  do
6:    $\text{swap}(P[i], P[j])$ 
7:    $i \leftarrow j$ 
8:    $j \leftarrow \text{indexMinChild}(P, i)$ 
9: end while
10: return minentry
```

Le istruzioni eseguite dal ciclo while sono dette **down-heap bubbling**.

Come prima, la **correttezza** deriva dalla *heap-order property estesa* e dal fatto che essa può essere violata solo dalle coppie in cui $P[i]$ è l'antenato.

Anche per quanto riguarda la **complessità**, un ragionamento analogo a quello fatto per il metodo `insert()` ci porta a concludere che essa sia $\Theta(\log n)$.

RIEPILOGO: LISTE VS HEAP

Ricapitoliamo allora quanto visto per la complessità dei metodi in base all'implementazione scelta:

IMPLEMENTAZIONE	Insert	min	removeMin
Lista non ordinata	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Lista ordinata	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Heap	$\Theta(\log n)$	$\Theta(1)$	$\Theta(\log n)$

COSTRUZIONE HEAP

Consideriamo un problema dato dalla seguente specifica:

- **INPUT**: Array P con n entry $P[0 \div n - 1]$.
- **OUTPUT**: Array P organizzato per rappresentare uno *heap*.

ALGORITMO IN-PLACE

Un algoritmo si dice **in-place** se usa $O(1)$ *memoria aggiuntiva* oltre a quella necessaria per l'input.

Alcune *soluzioni banali* possono essere:

1. *Ordinare* P in senso *non-decrescente* usando:
 - insertionSort: tempo $\Theta(n^2)$, *in-place*.
 - mergeSort: tempo $\Theta(n \log n)$, *non in-place*.
2. Trasferire le entry da P ad un *array di appoggio* Q e poi rimetterle in P invocando n volte insert(): tempo $\Theta(n \log n)$, *non in-place*.

APPROCCIO TOP-DOWN

L'idea è di eseguire $n - 1$ iterazioni successive mantenendo il seguente invariante alla fine di ciascuna iterazione j con $1 \leq j \leq n - 1$.

INVARIANTE

- $P[0 \div j]$ contiene le entry iniziali *riordinate* in modo da formare uno heap.
- $P[j + 1 \div n - 1]$ *immutato*.

Si tratta quindi di eseguire un **up-heap bubbling** da $P[j]$ in ciascuna iterazione j .

Algorithm top_down()

```
1: last ← n-1
2: for j ← 1 to n-1 do
3:   i ← j
4:   while i > 0 and P[⌊ $\frac{i-1}{2}$ ⌋].getKey() > P[i].getKey() do
5:     swap(P[i], P[⌊ $\frac{i-1}{2}$ ⌋])
6:     i ← P[⌊ $\frac{i-1}{2}$ ⌋]
7:   end while
8: end for
```

La **correttezza** discende direttamente dall'invariante presentato nell'idea.

COMPLESSITA'

Dimostriamo che la complessità è:

$$\Theta\left(\sum_{j=1}^{n-1} \log j\right)$$

- L'iterazione j consiste nell'eseguire un **up-heap bubbling** per posizionare $P[j]$ in $P[0 \div j]$, che richiede un numero di operazioni $\leq j$. Per cui troviamo banalmente che la complessità è $O\left(\sum_j \log j\right)$.
- L'istanza in cui P è ordinato in senso **decrescente** richiede $\log j$ operazioni per ogni iterazione con $1 \leq j \leq n-1$, quindi abbiamo anche che la complessità è $\Omega\left(\sum_j \log j\right)$.

Dimostriamo ora che vale:

CLAIM

$$\sum_{j=1}^{n-1} \log j \in \Theta(n \log n)$$

DIM.

PRIMA PARTE:

$$\sum_{j=1}^{n-1} \log j \leq \sum_{j=1}^{n-1} \log n = (n-1) \log n < n \log n$$

Per cui abbiamo che la somma considerata è $\in O(n \log n)$.

SECONDA PARTE:

$$\begin{aligned}\sum_{j=1}^{n-1} \log j &\geq \sum_{j=\lfloor \frac{n}{2} \rfloor}^{n-1} \log j \geq \sum_{j=\lfloor \frac{n}{2} \rfloor}^{n-1} \log \left(\left\lfloor \frac{n}{2} \right\rfloor \right) \\ &\geq \left\lfloor \frac{n}{2} \right\rfloor \log \left(\left\lfloor \frac{n}{2} \right\rfloor \right)\end{aligned}$$

Quindi la somma è anche $\in \Omega(n \log n)$ e concludiamo la dimostrazione $\square$.

APPROCCIO BOTTOM-UP

L'idea in questo caso è di eseguire $\lfloor \frac{n}{2} \rfloor$ iterazioni successive mantenendo il seguente invariante alla fine di ciascuna iterazione j con $\lfloor \frac{n-2}{2} \rfloor \geq j \geq 0$.

🔗 INVARIANTE

- $P[0 \div j - 1]$ *immutato*.
- $P[j \div n - 1]$ contiene le stesse entry iniziali, *riordinate* in modo da essere una *foresta di heap*.

Si tratta quindi di eseguire un **down-heap bubbling** da $P[j]$ in ciascuna iterazione j .

⚠️ ATTENZIONE

La scelta di partire da $j = \lfloor \frac{n-2}{2} \rfloor$ è dovuta al fatto che questo è l'indice del *nodo interno più a destra* nel penultimo livello: tutti i nodi di indice maggiore sono foglie, e quindi non è necessario eseguire il bubbling su di esse.

Algorithm bottom_up()

```

1: for  $j \leftarrow \lfloor (n-2)/2 \rfloor$  downto 0 do
2:    $i \leftarrow j$ 
3:    $k \leftarrow \text{indexMinChild}(P, i)$ 
4:   while  $k \neq \text{null}$  and  $P[i].\text{getKey}() > P[k].\text{getKey}()$  do
5:      $\text{swap}(P[i], P[k])$ 
6:      $i \leftarrow k$ 
7:      $k \leftarrow \text{indexMinChild}(P, i)$ 
8:   end while
9: end for
```

Anche in questo caso la **correttezza** discende direttamente dall'invariante riportato.

COMPLESSITA'

- Chiamiamo $t_{P[j]}$ il *costo del down-heap bubbling* a partire da $P[j]$.
- Per ogni nodo al livello i con $0 \leq i \leq h-1$ e $h = \lfloor \log_2 n \rfloor$ il down-heap bubbling costa $O(h-i)$, cioè ha un costo *proporzionale* alla *distanza dalle foglie*.
- 2^i nodi al livello i .

La complessità è quindi:

$$O\left(\sum_{j=0}^{\lfloor (n-2)/2 \rfloor} t_{P[j]}\right) = O\left(\sum_{i=0}^{h-1} 2^i (h-i)\right)$$

Dimostreremo ora due affermazioni (*Claim 1* e *Claim 2*) per concludere che la complessità è $O(n)$.

CLAIM 1

$$\sum_{l=1}^h l \left(\frac{1}{2}\right)^l < 3$$

✓ DIM.

Osserviamo che $\forall l \geq 1$ si ha:

$$l \left(\frac{1}{2}\right)^l \leq \left(\frac{3}{4}\right)^l$$

Che è facilmente dimostrabile per induzione.

Allora abbiamo:

$$\begin{aligned} \sum_{l=1}^h l \left(\frac{1}{2}\right)^l &\leq \sum_{l=1}^h \left(\frac{3}{4}\right)^l \\ &= \frac{(3/4)^{h+1} - (3/4)}{(3/4) - 1} = \frac{(3/4) - \overset{>0}{\cancel{(3/4)^{h+1}}}}{1/4} \\ &< \frac{3/4}{1/4} = 3 \end{aligned}$$

E concludiamo $\square$.

Procediamo ora con la seconda asserzione:

CLAIM 2

$$\sum_{i=0}^{h-1} 2^i h - i \in O(n)$$

✓ DIM.

Riscriviamo la sommatoria nel seguente modo:

$$2^h \sum_{i=0}^{h-1} \frac{2^i}{2^h} (h-i) = 2^h \sum_{i=0}^{h-1} \frac{h-i}{2^{h-i}}$$

Introduciamo a questo punto il cambio di variabile $l = h - i$, ottenendo:

$$2^h \sum_{l=1}^h l \left(\frac{1}{2}\right)^l$$

Per quanto visto sopra, abbiamo allora che la somma è

$$< 2^h \cdot 3$$

Ricordando che $h = \lfloor \log_2 n \rfloor$ troviamo anche che la quantità sopra è uguale a:

$$3 \lfloor \log_2 n \rfloor \in O(n)$$

E concludiamo la dimostrazione $\square$.

CONFRONTO APPROCCI

Per l'approccio **top-down** abbiamo:

- 2^i operazioni di costo $\Theta(i)$.
- Più *aumenta* la taglia del *livello* e più *aumenta anche* il *costo* dell'*up-heap bubbling*.
- Complessivamente: $\Theta(n \log n)$.

Per l'approccio **bottom-up**, invece:

- 2^i operazioni di costo $\Theta(\log n - i)$.
- Più *aumenta* la taglia del *livello* e più *diminuisce* il *costo* del *down-heap bubbling*.
- Complessivamente: $\Theta(n)$.

🔗 NOTA

Entrambe le soluzioni sono comunque implementabili senza utilizzare spazio aggiuntivo (*in-place*).

SORTING CON PRIORITY QUEUE

Sia $S = S[0], S[1], \dots, S[n-1]$ una **sequenza** di n chiavi **da ordinare**.

Vediamo l'algoritmo $\text{pqSort}(S)$ che si basa sulla seguente idea:

$$S \text{ disordinata} \xrightarrow{A} P \xrightarrow{B} S \text{ ordinata}$$

- Fase **A**: si inseriscono le n chiavi in una **priority queue** P una alla volta invocando il metodo `insert` (considerando le chiavi come entry).
- Fase **B**: si rimuovono le n chiavi da P una alla volta invocando il metodo `removeMin`.

Per quanto riguarda la complessità delle due fasi abbiamo:

1. Se P è una **lista non ordinata**.

- Fase **A**: $\Theta(n)$.
- Fase **B**: $\Theta(\sum_{i=1}^n i) \in \Theta(n^2)$ (**selection sort**).

2. Se P è una **lista ordinata**:

- Fase **A**: $\Theta(\sum_{i=1}^n i) \in \Theta(n^2)$ (**insertion sort**).
- Fase **B**: $\Theta(n)$.

3. Se P è un **heap** (su array):

- Fasi **A,B**: $\Theta(\sum_{i=1}^n \log i) \in \Theta(n \log n)$.
- Con costruzione **bottom-up** la fase A scende a $\Theta(n)$, mentre B rimane $\Theta(n \log n)$.

Quest'ultimo approccio è un algoritmo noto come **HeapSort**.

IMPLEMENTAZIONE IN-PLACE

OSS.

InsertionSort e SelectionSort possono essere implementati **in-place** in modo semplice.

Possiamo fare lo stesso per **HeapSort**?

L'idea è quella di implementare una variante di HeapSort **in-place** usando la **stessa sequenza** S come sequenza di **input**, **priority queue** e anche sequenza di **output**.

Per farlo utilizziamo un **max-heap** anziché un **min-heap**.

- Fase **A**: **riorganizza** $S[0 \div n-1]$ in modo che le chiavi rappresentino un **max-heap** (la chiave in un nodo interno è $\geq$ delle chiavi nei figli).
- Fase **B**: riorganizza $S[0 \div n-1]$ in modo che le chiavi risultino **ordinate**.

FASE A

La trasformazione di S in un max-heap è implementata come segue, considerando un metodo $\text{indexMaxChild}(S, i)$ che restituisce l'indice del figlio di $S[i]$ con *chiave massima* ($2i + 1$ o $2i + 2$) se $S[i]$ è interno, oppure null se $S[i]$ è foglia.

Algorithm faseA()

```
1: last ← n-1
2: for j ← ⌊(n-2)/2⌋ downto 0 do
3:   i ← j
4:   k ← indexMaxChild(P, i)
5:   while k ≠ null and S[i] < S[k] do
6:     swap(S[i], S[k])
7:     i ← k
8:     k ← indexMaxChild(S, i)
9:   end while
10: end for
```

FASE B

La fase B è basata su un ciclo *for* di $n - 1$ iterazioni ($j = 0, \dots, n - 1$) che mantiene il seguente invariante alla fine della j -esima iterazione:

🔗 INVARIANTE

- $S[0 \div n - j - 1]$ contiene le $n - j$ *chiavi più piccole* organizzate come **max-heap**. (Sequenza non ancora modificata)
- $S[n - j \div n - 1]$ contiene le j *chiavi più grandi* in *ordine crescente*. (Sequenza modificata)

Algorithm faseB()

```
1: last ← n-1
2: for j ← 1 to n-1 do
3:   swap( S[last], S[0] )
4:   last ← n-j-1
5:   i ← 0
6:   k ← indexMaxChild(S, i)
7:   while k ≠ null and S[i] < S[k] do
8:     swap(S[i], S[k])
9:     i ← k
10:    k ← indexMaxChild(S, i)
11:  end while
12: end for
```

L'algoritmo consiste quindi nel *scambiare la prima entry* (*massima* vista l'organizzazione) dalla porzione di sequenza che rappresenta l'heap con quella

presente in ultima posizione (si costruisce quindi la *sequenza ordinata* a partire *dalla coda*) e ripristinare la heap-order property con *down-heap bubbling*.

COMPLESSITA'

- **Fase A:** equivalente alla costruzione *bottom-up*, quindi $\Theta(n)$.
- **Fase B:** equivale all'esecuzione di $n - 1$ `removeMax()` da heap progressivamente più piccoli, per cui la complessità è $\Theta(\sum_i \log i) \in \Theta(n \log n)$

HEAP SORT

Concludiamo quindi che HeapSort è un algoritmo *in-place* e di complessità $\Theta(n \log n)$: è il miglior algoritmo di ordinamento che conosciamo fino a questo punto del corso.